OSCAR: Ontology-Guided Contrastive Learning for Job Matching via Signal-Aligned Representation

Olatayo Olukotun
North Carolina State University
Raleigh, North Carolina, United States
ojolukot@ncsu.edu

Habib Mohammed
North Carolina State University
Raleigh, North Carolina, United States
himohamm@ncsu.edu

Mulikatu Yakubu
North Carolina State University
Raleigh, North Carolina, United States
miyakubu@ncsu.edu

Kemafor Ogan
North Carolina State University
Raleigh, North Carolina, United States
kogan@ncsu.edu

Abstract

Job-candidate matching represents a critical challenge in modern recruitment, requiring systems to understand nuanced career progression patterns and professional qualifications. Existing approaches rely primarily on keyword matching or general-purpose neural encoders that treat all mismatches uniformly, failing to distinguish between candidates who are "almost qualified" (e.g., one career level away) versus those from completely unrelated domains. Moreover, current systems provide only confidence scores or opaque similarity rankings, offering little insight into why specific matches are recommended. While recent contrastive learning methods have shown promise for dense retrieval tasks, they employ domain-agnostic negative sampling strategies that ignore the rich structural knowledge encoded in professional ontologies like the European Skills, Competences, Qualifications and Occupations (ESCO) framework.

We present OSCAR, a framework that integrates the ESCO occupational skills ontology into contrastive learning for job-resume matching through two mechanisms: ontology-guided negative selection, which uses skill-level graph distances to construct informative hard, medium, and easy negatives; and ontology-aware sample weighting, which modulates per-sample loss contributions based on ESCO skill coverage quality. Through systematic ablation across data regimes, we find that ontology-guided negative selection is the primary driver of improvement. We further investigate incorporating ISCO occupational hierarchy codes as a complementary signal for negative selection, finding that ISCO group distance improves negative bucketing at full scale but adds noise at smaller data volumes. Experiments on a real-world job matching dataset with three-tier relevance labels demonstrate that OSCAR consistently outperforms vanilla InfoNCE on ranking metrics including AUC-ROC, Spearman correlation, and NDCG.

Conference acronym 'XX, Woodstock, NY

ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

CCS Concepts

• **Information systems** → *Language models.*

Keywords

Neurosymbolic Recommenders, Ontology-Guided Contrastive Learning, Job Matching

ACM Reference Format:
Olatayo Olukotun, Habib Mohammed, Mulikatu Yakubu, and Kemafor Ogan. 2026. OSCAR: Ontology-Guided Contrastive Learning for Job Matching via Signal-Aligned Representation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX).* ACM, New York, NY, USA, 11 pages. https://doi.org/XXXXXXX.XXXXXXX

1 Introduction

Job–candidate matching is a central challenge in modern recruitment. Systems must recognize nuanced career progression patterns, capture transferable skills, and produce *explainable* recommendations that multiple stakeholders can trust. In practice, successful matching often hinges on distinguishing candidates who are *almost qualified* (e.g., one career level away on the same ladder) from those in *unrelated domains.*

Manual, feature–engineered methods (e.g., APJFNN [**?**]) rely on explicit signals such as keyword overlap, seniority rules, and trajectory scores. While interpretable, these approaches are brittle, labor–intensive, and struggle to generalize beyond surface forms.

Dense retrieval methods (e.g., ConFit [37]) instead train dual encoders to embed resumes and jobs into a continuous vector space optimized with contrastive objectives such as InfoNCE [30], and retrieve candidates efficiently with ANN libraries like FAISS [**?**]. ConFit reports strong gains (e.g., up to 31% absolute improvement in nDCG@10) via textual augmentation and hard negatives mined from historically rejected pairs. However, dense systems typically *treat all mismatches uniformly*: for a *Junior Developer* position, a rejected *Senior Developer* and a rejected *Marketing Manager* are penalized equally, despite the former being much closer on a realistic career path. Moreover, paraphrase-based augmentation preserves wording but *does not encode* the semantic transformations inherent in career progression (e.g., junior → senior role perspectives).

A neurosymbolic AI perspective. We propose *ontology–guided contrastive learning*, a neurosymbolic framework that fuses structured knowledge with neural embeddings to address both performance and explainability. We extend the European Skills, Competences, Qualifications and Occupations (ESCO) ontology [**?**] with curated *career progression paths* in the CS/IT domain (e.g., *Junior Developer* → *Software Developer* → *Senior Developer* → *Tech Lead*). These paths supervise pair construction and *loss weighting* via explicit *career distance*. Adjacent roles on the same path (e.g., Junior→Senior Developer) are treated as *hard positives*; near–path but distinct roles (e.g., *Backend Developer* vs. *Data Engineer*) become *hard negatives*; unrelated roles (e.g., *Developer* vs. *Nurse*) remain *easy negatives*.

Why this matters for explainability. Grounding training in ESCO's structured concepts yields embeddings that remain aligned with interpretable career notions (levels, ladders, domains), enabling *inherent explainability* rather than post-hoc approximations. For example, our system can justify a ranking by citing (i) *CareerFit* (adjacent vs. distant levels on a path) and (ii) *SkillAlign* (coverage of ESCO skills), producing concise, auditable rationales for HR, managers, and candidates.

Beyond paraphrase: career–aware augmentation. Unlike text paraphrasing (e.g., "Experienced with Java" → "Proficient in the Java programming language"), our *progression* transformations reflect genuine growth. A junior-level bullet "*Wrote Python scripts to process logs*" may be transformed upward to "*Architected a Python-based data pipeline for production log analytics*," encoding responsibility and scope changes consistent with advancement constraints in ESCO.

Contributions. This paper makes the following contributions:

(1) **Neurosymbolic, ontology–guided contrastive framework.** We introduce a job–matching system that integrates ESCO [**?**] career paths directly into the contrastive objective, aligning dense embeddings with professional advancement structure.
(2) **Career–distance–weighted InfoNCE.** We extend InfoNCE [30] with distance–based weighting so that adjacent roles exert stronger pressure than distant mismatches, improving discrimination where hiring decisions are most subtle.
(3) **Career–aware augmentation.** We generate upward and downward role views along progression paths (hard positives) that capture authentic shifts in responsibility and scope beyond paraphrase.
(4) **Inherent explainability.** Because representations are trained against explicit career distances and ESCO skills, the system produces faithful, ontology-grounded explanations (e.g., *CareerFit*, *SkillAlign*) suitable for high-stakes hiring audits.
(5) **Scalable dense retrieval.** We retain the efficiency of dual encoders and ANN search (e.g., FAISS [**?**]) and demonstrate improvements over strong baselines including ConFit [37], with ablations confirming each component's contribution.

2 Background and Related Work

https://medium.com/ziprecruiter-tech/multimodal-learning-for-employment-marketplace-recommendation-ee67bdbede53 https://recsyshr.aau.dk/wp-content/uploads/2024/10/RecSysHR2024-paper_8.pdf

2.1 Job-Candidate Matching with Neural Approaches

Job-candidate matching is formulated as a retrieval problem where, given a candidate profile c and a set of job postings $\mathcal{J} = \{j_1, j_2, \ldots, j_n\}$, the goal is to rank jobs by their compatibility with the candidate. This differs from general document retrieval in its inherent asymmetry—job descriptions emphasize requirements and responsibilities while candidate profiles highlight achievements and capabilities—and the structured nature of career progression that governs professional compatibility.

Early approaches relied on keyword matching and traditional IR methods [9, 25], which suffered from vocabulary mismatch and inability to capture semantic relationships between skills and requirements. The emergence of neural approaches has addressed many limitations through learned representations. BERT-based methods [11, 12] demonstrated improvements by fine-tuning pre-trained language models on job-resume pairs, while domain-specific variants like JobBERT [11] showed the value of specialized recruitment training.

The success of dual-encoder architectures in dense retrieval [17, 35] inspired their adaptation to job matching. [18] introduced a foundational BERT-based approach using Siamese dual encoders with multi-task learning to jointly embed resumes and job descriptions. [21] proposed a bidirectional deep learning model with dual-stream LSTM architecture and attention mechanisms to evaluate person-job fit through aligned representations. [1] developed learning-to-rank approaches using pairwise and listwise ranking losses that share conceptual similarities with contrastive objectives.

More recently, explicit contrastive learning has been applied to job matching. Yan et al. [**?**] used InfoNCE loss to train dual encoders on historical job-resume application data, with positive pairs from matched applications and negative pairs from random sampling. Yu et al. [37] introduced ConFit, achieving up to 31% absolute improvement in nDCG@10 through data augmentation and strategic negative sampling from historically rejected pairs. However, ConFit's approach treats all negatives within rejection categories uniformly—a rejected senior developer application and a rejected marketing application receive equal treatment as hard negatives for a junior developer position, missing opportunities for graded supervision that respects professional advancement structures. Their text-based augmentation preserves linguistic meaning but cannot capture semantic transformations inherent in career progression.

Tang et al. [28] achieved state-of-the-art performance by combining candidate-job graphs with LLM-generated profiles through cross-modal contrastive learning. Career trajectory modeling has emerged as a complementary direction, with CareerBERT variants [10, 24] embedding resume histories and ESCO occupations for career path prediction. However, these approaches use shared encoders with uniform negative sampling and incorporate symbolic information only during post-hoc scoring, limiting structured explainability aligned with career progression logic.

While these neural methods demonstrate substantial performance improvements, they remain purely neural systems lacking

ontology grounding and fail to leverage structured career progression patterns that are fundamental to professional compatibility assessment.

2.2 Contrastive Learning and Negative Sampling

Contrastive learning has emerged as a dominant paradigm for learning representations that capture semantic relationships. The InfoNCE objective [30] trains encoders to maximize similarity between positive pairs while minimizing similarity to negative examples:

$$\mathcal{L} = -\log \frac{\exp(\text{sim}(z_i, z_i^+)/\tau)}{\exp(\text{sim}(z_i, z_i^+)/\tau) + \sum_j \exp(\text{sim}(z_i, z_j^-)/\tau)} \quad (1)$$

where z_i is the anchor representation, z_i^+ is the positive example, z_j^- are negative examples, and τ is a temperature parameter.

The effectiveness of contrastive learning heavily depends on negative sampling strategies. Standard approaches either sample negatives uniformly from the training batch [6, 14] or employ similarity-based hard negative mining [16, 23]. Recent work has explored uncertainty-based sampling [7] and representativeness-aware selection [27], but these methods rely on learned similarity metrics to identify challenging examples, risking selection of false hard negatives from the same semantic class.

In structured domains like career matching, such domain-agnostic approaches fail to leverage established knowledge about meaningful distinctions. A junior developer matched against a senior developer position represents a fundamentally different type of mismatch than the same candidate matched against a marketing role, yet current methods treat these scenarios identically during training.

2.3 Neurosymbolic Integration in Recommendation

Traditional ontology-based recommender systems rely entirely on symbolic reasoning over structured knowledge representations [4, 13, 20]. While these approaches provide inherent explainability through explicit reasoning chains, they suffer from scalability constraints, brittleness when encountering novel scenarios, and inability to capture semantic similarity beyond exact symbolic matches. These limitations have motivated neurosymbolic approaches that combine neural pattern recognition with structured symbolic reasoning.

Neurosymbolic approaches aim to combine the pattern recognition capabilities of neural networks with the structured reasoning of symbolic systems. In recommendation contexts, knowledge graphs have been integrated with neural architectures through various strategies [32, 33]. Traditional approaches learn entity embeddings using methods like TransE [3] or ComplEx [29], then incorporate these as auxiliary features in neural recommenders [31, 38].

More recent work employs graph neural networks to model entity relationships directly [33, 36], enabling end-to-end learning over knowledge graph structure. However, these approaches typically operate as separate modules from the main retrieval objective, requiring independent training phases and failing to integrate structured knowledge into core contrastive learning processes.

The European Skills, Competences, Qualifications and Occupations (ESCO) framework [8] provides a standardized ontology encoding skill hierarchies and occupational relationships relevant to job matching. While ESCO has been leveraged for skill extraction and job classification [2, 11], existing approaches use it primarily for feature engineering or post-processing rather than integrating its hierarchical structure directly into the learning objective. Zhang et al. [39] demonstrated how symbolic taxonomies can enhance neural representations through ESCOXLM-R, a multilingual transformer that internalizes ESCO knowledge via domain-adaptive pretraining and relation prediction tasks.

However, existing integration approaches represent a missed opportunity to leverage decades of curated expertise about career progression patterns directly within contrastive learning objectives, limiting their ability to provide structured explanations grounded in professional advancement logic.

2.4 Explainable Recommendation Systems

Explainability in recommendation systems has become increasingly important, particularly in high-stakes domains like hiring where decisions require justification to multiple stakeholders [5, 40]. Current approaches to explainable recommendation typically fall into two categories: post-hoc explanation methods that attempt to interpret trained models [19, 22], and inherently interpretable models that constrain architectures to maintain transparency [5, 40].

In job matching contexts, existing systems predominantly provide confidence scores or similarity rankings without meaningful interpretation [26]. Post-hoc attention visualization methods [15, 34] have been applied but suffer from reliability issues and fail to provide the structured, actionable feedback needed in professional contexts. Moreover, these approaches cannot guarantee faithfulness to the actual decision process, particularly problematic in hiring scenarios requiring auditability and bias detection.

The integration of structured domain knowledge presents an opportunity for inherent explainability, where learned representations maintain direct connections to interpretable domain concepts, enabling explanation generation that leverages established professional vocabularies and career progression logic.

3 Proposed Dual-Purposed Neurosymbolic Framework

3.1 Problem Formulation

Let $\mathcal{R} = \{r_1, r_2, \ldots, r_n\}$ represent the set of candidate resumes and $\mathcal{J} = \{j_1, j_2, \ldots, j_m\}$ denote the set of job postings. Each resume r_i and job posting j_k are represented as structured text documents. The training data consists of labeled pairs $\mathcal{D} = \{(r_i, j_k, y_{ik})\}$ where $y_{ik} \in \{0, 1\}$ indicates whether candidate r_i received an interview for job j_k.

Our goal is to learn a matching function $f : \mathcal{R} \times \mathcal{J} \to \mathbb{R}$ that computes compatibility scores while maintaining explicit mappings to career progression concepts for explainability. Unlike standard approaches that treat all mismatches uniformly, our method leverages career progression structure encoded in an extended ESCO ontology $\mathcal{G}_{career}$ to provide graduated supervision.

4 Methodology

4.1 System Architecture

We present a neurosymbolic approach for explainable job-candidate matching that integrates career progression knowledge directly into contrastive learning objectives. Our framework addresses the limitations of existing approaches through three key innovations: (1) pathway-aware contrastive learning with ontology-guided negative sampling, (2) career-aware data augmentation that respects professional advancement structures, and (3) inherently explainable representations that enable structured career-based justifications. The architecture achieves neurosymbolic integration through three key mechanisms: (1) **Training-Inference Consistency** - identical career distance weights used in both contrastive training and similarity scoring, (2) **Structured Supervision** - career graph topology provides graduated negative sampling during training and interpretable reasoning during inference, and (3) **Dual Representation** - each profile maintains both dense embeddings for semantic similarity and structured attributes for symbolic reasoning. Our framework operates through two phases as illustrated in Figure 1. The offline training phase learns career-aware dual encoders through pathway-guided contrastive learning that integrates ESCO-derived career progression knowledge directly into the InfoNCE objective. The online matching phase combines efficient neural retrieval with symbolic career distance reasoning to produce ranked results and structured explanations. This design addresses limitations of pure neural approaches (lack of interpretability) and pure symbolic systems (scalability constraints and semantic inflexibility) while enabling scalable matching with inherent explainability through career progression concepts.

4.2 Dual-Encoder Architecture with Career Integration

Our framework employs specialized dual encoders $\mathcal{E}_r : \mathcal{R} \to \mathbb{R}^d$ and $\mathcal{E}_j : \mathcal{J} \to \mathbb{R}^d$ optimized for resume and job content respectively. Unlike shared-encoder approaches that must accommodate both resume achievement descriptions and job requirement specifications within a single parameter space, dual encoders enable domain-specific optimization while maintaining embedding space compatibility for similarity computation.

The final similarity function integrates neural representations with symbolic career reasoning:

$$\text{sim}(r, j) = \frac{\mathcal{E}_r(r)^T \mathcal{E}_j(j)}{\|\mathcal{E}_r(r)\| \|\mathcal{E}_j(j)\|} \cdot w_d(\ell(r), \ell(j)) \tag{2}$$

where $w_d(\ell(r), \ell(j))$ applies the same distance weighting used during training, ensuring training-inference consistency critical for explanation faithfulness.

4.3 Career Progression Graph Construction

We extend the ESCO occupation framework with curated advancement relationships to create a comprehensive career progression graph $G_{\text{career}} = (V, E)$. Vertices represent professional roles while edges encode typical advancement patterns. For computer science roles, we define structured pathways such as:

$$\text{Junior Developer} \to \text{Software Developer} \to \text{Senior Developer} \to \text{Tech Lead} \to \text{Engineering Manager} \tag{3}$$

with parallel specialization tracks (e.g., Senior Developer → Principal Engineer → Staff Engineer). The graph enables computation of career distance $d_{\text{career}}(\ell_1, \ell_2)$ as the shortest path between levels, providing structured guidance for both negative sampling and loss weighting.

4.4 Pathway-Aware Contrastive Learning

Standard contrastive learning assumes all negatives are equally informative, which fails in career domains. When training on a *Junior Developer* role, both a *Senior Developer* and a *Nurse* serve as negatives, yet the former provides substantially more informative supervision for professional boundary learning. To address this, we integrate ontological knowledge about career-progression structure into both negative sampling and loss weighting.

Ontology-guided negative structure. We leverage G_{career} to construct graduated negative sets. For a positive pair (r^+, j^+):

$$N_{\text{1-hop}} = \{j \in J | d_{\text{career}}(\ell(j^+), \ell(j)) = 1\}, \tag{4}$$

$$N_{\text{2-hop}} = \{j \in J | d_{\text{career}}(\ell(j^+), \ell(j)) = 2\}, \tag{5}$$

$$N_{\text{distant}} = \{j \in J | d_{\text{career}}(\ell(j^+), \ell(j)) \geq 5\}. \tag{6}$$

Here $\ell(\cdot)$ maps resumes and job postings to their inferred career levels within the progression graph (e.g., Junior Developer, Senior Developer, Data Scientist) This creates three negative categories: adjacent career levels (hard negatives providing fine-grained supervision), nearby levels (moderate negatives), and distant roles (easy negatives establishing clear boundaries). All negatives are weighted continuously based on career distance rather than categorical domain membership.

4.5 Career-Aware Augmentation

We enrich the positive set through career-aware augmentations that generate semantically valid transformations reflecting professional growth patterns, unlike standard paraphrasing that preserves only linguistic meaning.

Progression-based transformations. Given a resume r at level ℓ, we create augmented views representing the same professional background from adjacent career perspectives:

$$r^{(\ell+1)} = T_{\text{up}}(r, G_{\text{career}}), \tag{7}$$

$$r^{(\ell-1)} = T_{\text{down}}(r, G_{\text{career}}). \tag{8}$$

The upward transformation T_{up} enhances responsibility descriptions and technical depth, while T_{down} simplifies scope while retaining core competencies. For example:

> **Original:** "Wrote Python scripts to process logs"
> T_{up} **(Senior view):** "Architected a Python-based data pipeline for production log analytics"
> T_{down} **(Intern view):** "Assisted in writing Python scripts for routine log processing"

These augmented views serve as additional positives, enabling the model to learn invariances across career presentation styles while respecting professional advancement semantics.

4.6 Career-Distance-Weighted Contrastive Loss

We integrate structured negatives and augmented positives through a career-aware extension of InfoNCE that applies graduated penalties based on career graph topology:

$$L_{career} = -\log \frac{f(r^+, j^+)}{f(r^+, j^+) + \sum_{j^- \in N} w_d(d_{career}(\ell(j^+), \ell(j^-))) \cdot f(r^+, j^-)}, \tag{9}$$

where $f(x_1, x_2) = \exp(\text{sim}(x_1, x_2)/\tau)$ and N is the union of structured negative sets above. The weight $w_d(j^+, j^-)$ scales penalties by career distance:

$$w_d(d_{career}) = \begin{cases} 1.0 + \alpha \cdot \max(0, 1.0 - \frac{d_{career}}{d_{max}}) & \text{if career pairs} \\ 1.0 & \text{if explicit negatives} \end{cases} \tag{10}$$

where $\alpha = 2.0$ is the pathway emphasis parameter that controls the degree of distance-based weighting, $d_{max} = 10.0$ represents the maximum meaningful career distance within our CS/IT domain, and d_{career} is the shortest path distance between career levels in the progression graph. This formulation ensures smooth weight transitions rather than discrete jumps, providing graduated penalties that reflect the continuous nature of career progression relationships. This graduated weighting directly addresses the uniform negative treatment problem identified earlier: adjacent-level negatives (Senior Developer vs Junior Developer position) receive weight 2.8, providing a strong learning signal for fine-grained career distinctions, while distant negatives (Marketing Manager vs Junior Developer) receive weight 1.0, ensuring adequate separation without dominating the loss function.

Augmented views are incorporated by extending the loss to include career-level invariance:

$$\mathcal{L}_{\text{augmented}} = g(r) + \lambda_1 g(r^{(\ell+1)}) + \lambda_2 g(r^{(\ell-1)}) \tag{11}$$

Here function g is defined as $g(x) \to \mathcal{L}_{\text{career}}(x, j^+, \mathcal{N})$, while λ_1 and λ_2 control the contribution of augmented views.

Following ConFit's bidirectional insight, we employ symmetric training:

$$L_{\text{total}} = L^{R \to J}_{\text{augmented}} + L^{J \to R}_{\text{augmented}} \tag{12}$$

enabling effective ranking in both resume-to-job and job-to-resume directions while maintaining career-structure awareness throughout.

4.7 Matching Framework

Given a resume r or job posting j as raw text, the system first extracts structured professional information alongside generating neural embeddings. The complete matching pipeline integrates neural embeddings with structured career reasoning through four key stages: query processing, initial retrieval via FAISS, career-aware reranking, and explanation generation. This unified approach maintains training-inference consistency by applying identical distance weighting functions throughout.

4.7.1 Professional Information Extraction. We employ domain-specific extractors to identify key professional attributes:

$$\text{Skills}(r) = \text{SkillExtractor}(r, \mathcal{T}_{ESCO}) \tag{13}$$

$$\ell(r) = \text{CareerClassifier}(r, \mathcal{G}_{career}) \tag{14}$$

$$\text{Experience}(r) = \text{ExperienceParser}(r) \tag{15}$$

where $\mathcal{T}_{ESCO}$ represents the ESCO skill taxonomy and $\mathcal{G}_{career}$ is our career progression graph. The skill extractor maps textual skill mentions to standardized ESCO categories, while the career classifier infers professional level based on responsibility indicators, experience duration, and title patterns.

For job postings, we additionally parse requirement structures:

$$\text{Required}(j) = \text{RequirementExtractor}(j, \text{critical}) \tag{16}$$

$$\text{Preferred}(j) = \text{RequirementExtractor}(j, \text{preferred}) \tag{17}$$

4.7.2 Neural Embedding Generation. Parallel to structured extraction, we generate dense representations using trained dual encoders:

$$\mathbf{e}_r = \mathcal{E}_r(r) \in \mathbb{R}^d \tag{18}$$

$$\mathbf{e}_j = \mathcal{E}_j(j) \in \mathbb{R}^d \tag{19}$$

where $\mathcal{E}_r$ and $\mathcal{E}_j$ are specialized encoders optimized for resume and job content respectively through pathway-aware contrastive learning. The dual-encoder architecture enables domain-specific optimization while maintaining compatibility for similarity computation.

4.8 Efficient Candidate Retrieval with FAISS

Direct similarity computation over large candidate pools (potentially millions of resumes or jobs) is computationally prohibitive. We employ Facebook AI Similarity Search (FAISS) [**?**] for efficient approximate nearest neighbor retrieval.

4.8.1 Index Construction. For a database of N items (resumes or jobs), we pre-compute embeddings and construct FAISS indices:

$$\mathcal{I}_{\text{jobs}} = \text{FAISS-IVF}(\{\mathbf{e}_{j_1}, \mathbf{e}_{j_2}, \ldots, \mathbf{e}_{j_N}\}) \tag{20}$$

$$\mathcal{I}_{\text{resumes}} = \text{FAISS-IVF}(\{\mathbf{e}_{r_1}, \mathbf{e}_{r_2}, \ldots, \mathbf{e}_{r_M}\}) \tag{21}$$

We use Inverted File (IVF) indexing with Product Quantization for memory efficiency, enabling sub-linear search complexity over large databases.

4.8.2 Initial Candidate Retrieval. Given a query embedding $\mathbf{e}_q$, FAISS retrieval returns the top-k most similar candidates:

$$C_{\text{initial}} = \text{FAISS-Search}(\mathcal{I}, \mathbf{e}_q, k = \alpha \cdot k_{\text{final}}) \tag{22}$$

where $\alpha > 1$ (typically 2-3) ensures we retrieve more candidates than needed for final ranking. This over-retrieval compensates for potential ranking changes during career-aware re-scoring.

The FAISS retrieval provides candidate set $C_{\text{initial}} = \{(c_i, s_i^{\text{base}})\}$ where c_i represents candidate items and s_i^{base} denotes base cosine similarities from the approximate search.

4.9 Career-Aware Similarity Scoring and Re-ranking

This stage constitutes the core technical contribution: transforming base similarities into career-progression-aware compatibility scores that reflect professional advancement patterns.

4.9.1 Career Distance Computation. For each candidate pair $(query, candidate)$, we compute career distance using the progression graph:

$$d_{\text{career}}(\ell_q, \ell_c) = \text{ShortestPath}(\mathcal{G}_{\text{career}}, \ell_q, \ell_c) \tag{23}$$

where ℓ_q and ℓ_c represent the career levels of query and candidate respectively. The shortest path distance captures the minimum advancement steps required between professional positions.

4.9.2 Distance-Weighted Similarity Computation. We apply career distance weighting to base similarities, incorporating domain knowledge directly into the ranking function:

$$s_{\text{final}}(q, c) = s_{\text{base}}(q, c) \cdot w_d(d_{\text{career}}(\ell_q, \ell_c)) \tag{24}$$

4.9.3 Final Ranking. Candidates are re-ranked by career-aware similarity scores:

$$\text{Ranking}(q) = \text{Sort}(\{(c_i, s_{\text{final}}(q, c_i))\}_{i=1}^{|C_{\text{initial}}|}, \text{descending}) \tag{25}$$

This produces the final ranked list where matches that represent natural career progressions are prioritized over those requiring significant professional transitions.

4.10 Basic Explanation Generation

To provide transparency into matching decisions, we generate structured explanations that highlight key factors influencing compatibility scores.

4.10.1 Factor Analysis. For each match (q, c), we analyze primary compatibility dimensions:

$$\text{SkillAlignment}(q, c) = \frac{|\text{Skills}(q) \cap \text{Required}(c)|}{|\text{Required}(c)|} \tag{26}$$

$$\text{CareerFit}(q, c) = f(d_{\text{career}}(\ell_q, \ell_c), w_d(\cdot)) \tag{27}$$

$$\text{ExperienceMatch}(q, c) = g(\text{Experience}(q), \text{Requirements}(c)) \tag{28}$$

These metrics quantify skill coverage, career level alignment, and experience compatibility respectively.

4.10.2 Explanation Synthesis. Basic explanations are generated by combining factor analysis with simple templates:

$$\text{Explanation}(q, c) = \text{Template}(\text{SkillAlignment}, \text{CareerFit}, s_{\text{final}}) \tag{29}$$

Example output: *"Strong match (score: 0.82). Candidate demonstrates 5/6 required skills including Python and system design. Career level alignment: Senior Developer position matches candidate's experience level."*

This approach provides interpretable justifications for matching decisions while avoiding the complexity of comprehensive explanation frameworks that would require separate evaluation methodologies.

4.11 Computational Complexity

The complete matching pipeline operates with the following complexity characteristics:

- **Embedding Generation**: $O(L \cdot d)$ where L is input length and d is embedding dimension
- **FAISS Retrieval**: $O(\log N + k)$ for approximate search over N items
- **Career-Aware Re-ranking**: $O(k)$ for career distance computation and weighting
- **Total Query Time**: $O(L \cdot d + \log N + k)$

This enables real-time matching over large databases while incorporating structured career knowledge that would be prohibitive with exhaustive similarity computation.

The framework successfully integrates neural representation learning with symbolic career progression knowledge, enabling both scalable matching performance and basic interpretability without requiring complex explanation evaluation methodologies.

5 Experiments

5.1 Experimental Setup

5.2 Resume and Job Description Dataset

5.2.1 Resume Datasets.

- Kaggle Dataset
- JobHop Dataset

5.2.2 Job Posting Datasets.

-

6 Results and Discussion

By fine-tuning our BERT model with a ground-truthed dataset enriched with both ESCO and user-defined predicates, we transform a blackbox moel into a glass-box system, yielding a job recommendation framewwork that is accurate, robust, explainable, and trustworthy.

7 Conclusion and Future work

Limitations and Future Work. Our approach is currently evaluated within the CS/IT domain, limiting generalizability claims to other professional fields with different career progression patterns. The career graph construction relies on manual curation of advancement relationships, which may not capture all valid transition paths or reflect evolving industry structures. While our basic explanation framework provides interpretable career-distance reasoning, more sophisticated explanation generation incorporating detailed skill gap analysis and personalized career advice represents a natural extension. Future work should explore automated career graph learning from large-scale professional networks, cross-domain transfer to other industries, and integration with dynamic career trajectory modeling to account for emerging roles and non-traditional career paths.

8 Title Information

The title of your work should use capital letters appropriately - https://capitalizemytitle.com/ has useful rules for capitalization. Use the `title` command to define the title of your work. If your work has a subtitle, define it with the `subtitle` command. Do not insert line breaks in your title.

If your title is lengthy, you must define a short version to be used in the page headers, to prevent overlapping text. The `title` command has a "short title" parameter:

```
\title[short title]{full title}
```

9 Authors and Affiliations

Each author must be defined separately for accurate metadata identification. As an exception, multiple authors may share one affiliation. Authors' names should not be abbreviated; use full first names wherever possible. Include authors' e-mail addresses whenever possible.

Grouping authors' names or e-mail addresses, or providing an "e-mail alias," as shown below, is not acceptable:

```
\author{Brooke Aster, David Mehldau}
\email{dave,judy,steve@university.edu}
\email{firstname.lastname@phillips.org}
```

The `authornote` and `authornotemark` commands allow a note to apply to multiple authors — for example, if the first two authors of an article contributed equally to the work.

If your author list is lengthy, you must define a shortened version of the list of authors to be used in the page headers, to prevent overlapping text. The following command should be placed just after the last `\author{}` definition:

```
\renewcommand{\shortauthors}{McCartney, et al.}
```

Omitting this command will force the use of a concatenated list of all of the authors' names, which may result in overlapping text in the page headers.

The article template's documentation, available at https://www.acm.org/publications/proceedings-template, has a complete explanation of these commands and tips for their effective use.

Note that authors' addresses are mandatory for journal articles.

10 Rights Information

Authors of any work published by ACM will need to complete a rights form. Depending on the kind of work, and the rights management choice made by the author, this may be copyright transfer, permission, license, or an OA (open access) agreement.

Regardless of the rights management choice, the author will receive a copy of the completed rights form once it has been submitted. This form contains LaTeX commands that must be copied into the source document. When the document source is compiled, these commands and their parameters add formatted text to several areas of the final document:

- the "ACM Reference Format" text on the first page.
- the "rights management" text on the first page.
- the conference information in the page header(s).

Rights information is unique to the work; if you are preparing several works for an event, make sure to use the correct set of commands with each of the works.

The ACM Reference Format text is required for all articles over one page in length, and is optional for one-page articles (abstracts).

11 CCS Concepts and User-Defined Keywords

Two elements of the "acmart" document class provide powerful taxonomic tools for you to help readers find your work in an online search.

The ACM Computing Classification System — https://www.acm.org/publications/class-2012 — is a set of classifiers and concepts that describe the computing discipline. Authors can select entries from this classification system, via https://dl.acm.org/ccs/ccs.cfm, and generate the commands to be included in the LaTeX source.

User-defined keywords are a comma-separated list of words and phrases of the authors' choosing, providing a more flexible way of describing the research being presented.

CCS concepts and user-defined keywords are required for for all articles over two pages in length, and are optional for one- and two-page articles (or abstracts).

12 Sectioning Commands

Your work should use standard LaTeX sectioning commands: `\section`, `\subsection`, `\subsubsection`, `\paragraph`, and `\subparagraph`. The sectioning levels up to `\subsusection` should be numbered; do not remove the numbering from the commands.

Simulating a sectioning command by setting the first word or words of a paragraph in boldface or italicized text is **not allowed.**

Below are examples of sectioning commands.

12.1 Subsection

This is a subsection.

12.1.1 *Subsubsection.* This is a subsubsection.

Paragraph. This is a paragraph.
Subparagraph This is a subparagraph.

13 Tables

The "`acmart`" document class includes the "`booktabs`" package — https://ctan.org/pkg/booktabs — for preparing high-quality tables.

Table captions are placed *above* the table.

Because tables cannot be split across pages, the best placement for them is typically the top of the page nearest their initial cite. To ensure this proper "floating" placement of tables, use the environment **table** to enclose the table's contents and the table caption. The contents of the table itself must go in the **tabular** environment, to be aligned properly in rows and columns, with the desired horizontal and vertical rules. Again, detailed instructions on **tabular** material are found in the *LaTeX User's Guide.*

Immediately following this sentence is the point at which Table 1 is included in the input file; compare the placement of the table here with the table in the printed output of this document.

To set a wider table, which takes up the whole width of the page's live area, use the environment **table*** to enclose the table's contents and the table caption. As with a single-column table, this wide table will "float" to a location deemed more desirable. Immediately following this sentence is the point at which Table 2 is included in

Table 1: Frequency of Special Characters

Non-English or Math	Frequency	Comments
Ø	1 in 1,000	For Swedish names
π	1 in 5	Common in math
\$	4 in 5	Used in business
Ψ_1^2	1 in 40,000	Unexplained usage

the input file; again, it is instructive to compare the placement of the table here with the table in the printed output of this document.

Always use midrule to separate table header rows from data rows, and use it only for this purpose. This enables assistive technologies to recognise table headers and support their users in navigating tables more easily.

14 Math Equations

You may want to display math equations in three distinct styles: inline, numbered or non-numbered display. Each of the three are discussed in the next sections.

14.1 Inline (In-text) Equations

A formula that appears in the running text is called an inline or in-text formula. It is produced by the **math** environment, which can be invoked with the usual `\begin . . . \end` construction or with the short form `$ . . . $`. You can use any of the symbols and structures, from α to ω, available in LaTeX [**?**]; this section will simply show a few examples of in-text equations in context. Notice how this equation: $\lim_{n\to\infty} x = 0$, set here in in-line math style, looks slightly different when set in display style. (See next section).

14.2 Display Equations

A numbered display equation—one set off by vertical space from the text and centered horizontally—is produced by the **equation** environment. An unnumbered display equation is produced by the **displaymath** environment.

Again, in either environment, you can use any of the symbols and structures available in LaTeX; this section will just give a couple of examples of display equations in context. First, consider the equation, shown as an inline equation above:

$$\lim_{n\to\infty} x = 0 \tag{30}$$

Notice how it is formatted somewhat differently in the **displaymath** environment. Now, we'll enter an unnumbered equation:

$$\sum_{i=0}^{\infty} x + 1$$

and follow it with another numbered equation:

$$\sum_{i=0}^{\infty} x_i = \int_0^{\pi+2} f \tag{31}$$

just to demonstrate LaTeX's able handling of numbering.

15 Figures

The "`figure`" environment should be used for figures. One or more images can be placed within a figure. If your figure contains third-party material, you must clearly identify it as such, as shown in the example below.

Figure 1: 1907 Franklin Model D roadster. Photograph by Harris & Ewing, Inc. [Public domain], via Wikimedia Commons. (https://goo.gl/VLCRBB).

Your figures should contain a caption which describes the figure to the reader.

Figure captions are placed *below* the figure.

Every figure should also have a figure description unless it is purely decorative. These descriptions convey what's in the image to someone who cannot see it. They are also used by search engine crawlers for indexing images, and when images cannot be loaded.

A figure description must be unformatted plain text less than 2000 characters long (including spaces). **Figure descriptions should not repeat the figure caption – their purpose is to capture important information that is not already provided in the caption or the main text of the paper.** For figures that convey important and complex new information, a short text description may not be adequate. More complex alternative descriptions can be placed in an appendix and referenced in a short figure description. For example, provide a data table capturing the information in a bar chart, or a structured list representing a graph. For additional information regarding how best to write figure descriptions and why doing this is so important, please see https://www.acm.org/publications/taps/describing-figures/.

15.1 The "Teaser Figure"

A "teaser figure" is an image, or set of images in one figure, that are placed after all author and affiliation information, and before the body of the article, spanning the page. If you wish to have such a figure in your article, place the command immediately before the `\maketitle` command:

```
\begin{teaserfigure}
  \includegraphics[width=\textwidth]{sampleteaser}
  \caption{figure caption}
  \Description{figure description}
\end{teaserfigure}
```

16 Citations and Bibliographies

The use of BibTeX for the preparation and formatting of one's references is strongly recommended. Authors' names should be complete — use full first names ("Donald E. Knuth") not initials ("D. E. Knuth") — and the salient identifying features of a reference should be included: title, year, volume, number, pages, article DOI, etc.

The bibliography is included in your source document with these two commands, placed just before the `\end{document}` command:

```
\bibliographystyle{ACM-Reference-Format}
\bibliography{bibfile}
```

Table 2: Some Typical Commands

Command	A Number	Comments
`\author`	100	Author
`\table`	300	For tables
`\table*`	400	For wider tables

where "`bibfile`" is the name, without the "`.bib`" suffix, of the BibTEX file.

Citations and references are numbered by default. A small number of ACM publications have citations and references formatted in the "author year" style; for these exceptions, please include this command in the **preamble** (before the command "`\begin{document}`") of your LATEX source:

```
\citestyle{acmauthoryear}
```

Some examples. A paginated journal article [**?**], an enumerated journal article [**?**], a reference to an entire issue [**?**], a monograph (whole book) [**?**], a monograph/whole book in a series (see 2a in spec. document) [**?**], a divisible-book such as an anthology or compilation [**?**] followed by the same example, however we only output the series if the volume number is given [**?**] (so Editor00a's series should NOT be present since it has no vol. no.), a chapter in a divisible book [**?**], a chapter in a divisible book in a series [**?**], a multi-volume work as book [**?**], a couple of articles in a proceedings (of a conference, symposium, workshop for example) (paginated proceedings article) [**? ?**], a proceedings article with all possible elements [**?**], an example of an enumerated proceedings article [**?**], an informally published work [**?**], a couple of preprints [**? ?**], a doctoral dissertation [**?**], a master's thesis: [**?**], an online document / world wide web resource [**? ? ?**], a video game (Case 1) [**?**] and (Case 2) [**?**] and [**?**] and (Case 3) a patent [**?**], work accepted for publication [**?**], 'YYYYb'-test for prolific author [**?**] and [**?**]. Other cites might contain 'duplicate' DOI and URLs (some SIAM articles) [**?**]. Boris / Barbara Beeton: multi-volume works as books [**?**] and [**?**]. A presentation [**?**]. An article under review [**?**]. A couple of citations with DOIs: [**? ?**]. Online citations: [**? ? ?**]. Artifacts: [**?**] and [**?**].

17 Acknowledgments

Identification of funding sources and other support, and thanks to individuals and groups that assisted in the research and the preparation of the work should be included in an acknowledgment section, which is placed just before the reference section in your document.

This section has a special environment:

```
\begin{acks}
...
\end{acks}
```

so that the information contained therein can be more easily collected during the article metadata extraction phase, and to ensure consistency in the spelling of the section heading.

Authors should not prepare this section as a numbered or unnumbered `\section`; please use the "`acks`" environment.

18 Appendices

If your work needs an appendix, add it before the "`\end{document}`" command at the conclusion of your source document.

Start the appendix with the "`appendix`" command:

```
\appendix
```

and note that in the appendix, sections are lettered, not numbered. This document has two appendices, demonstrating the section and subsection identification method.

19 Multi-language papers

Papers may be written in languages other than English or include titles, subtitles, keywords and abstracts in different languages (as a rule, a paper in a language other than English should include an English title and an English abstract). Use `language=...` for every language used in the paper. The last language indicated is the main language of the paper. For example, a French paper with additional titles and abstracts in English and German may start with the following command

```
\documentclass[sigconf, language=english, language=german,
               language=french]{acmart}
```

The title, subtitle, keywords and abstract will be typeset in the main language of the paper. The commands `\translatedXXX`, `XXX` begin title, subtitle and keywords, can be used to set these elements in the other languages. The environment `translatedabstract` is used to set the translation of the abstract. These commands and environment have a mandatory first argument: the language of the second argument. See `sample-sigconf-i13n.tex` file for examples of their usage.

20 SIGCHI Extended Abstracts

The "`sigchi-a`" template style (available only in LATEX and not in Word) produces a landscape-orientation formatted article, with a wide left margin. Three environments are available for use with the "`sigchi-a`" template style, and produce formatted output in the margin:

`sidebar`: Place formatted text in the margin.
`marginfigure`: Place a figure in the margin.
`margintable`: Place a table in the margin.

Acknowledgments

To Robert, for the bagels and explaining CMYK and color spaces.

References

[1] Qingyao Ai, Keping Bi, Jiafeng Guo, and W Bruce Croft. 2018. Learning a deep listwise context model for ranking refinement. In *The 41st international ACM SIGIR conference on research & development in information retrieval*. 135–144.

[2] Chantal Amrhein and Simon Clematide. 2018. Supervised OCR error detection and correction using statistical and neural machine translation methods. *Journal for Language Technology and Computational Linguistics (JLCL)* 33, 1 (2018), 49–76.

[3] Antoine Bordes, Nicolas Usunier, Alberto Garcia-Duran, Jason Weston, and Oksana Yakhnenko. 2013. Translating embeddings for modeling multi-relational data. *Advances in neural information processing systems* 26 (2013).

[4] Iván Cantador, Alejandro Bellogín, and Pablo Castells. 2008. A multilayer ontology-based hybrid recommendation model. In *AI Communications*, Vol. 21. IOS Press, 203–210.

[5] Nan Chen, Xiaochan Dong, Uday Lokala, and Yongfeng Zhang. 2018. Explainable recommendation via multi-task learning in opinionated text data. *The 41st international ACM SIGIR conference on research & development in information retrieval* (2018), 165–174.

[6] Ting Chen, Simon Kornblith, Mohammad Norouzi, and Geoffrey Hinton. 2020. A simple framework for contrastive learning of visual representations. *International conference on machine learning* (2020), 1597–1607.

[7] Chia-Yuan Chuang, Joshua Robinson, Yen-Chen Lin, Antonio Torralba, and Stefanie Jegelka. 2020. Debiased contrastive learning. *Advances in neural information processing systems* 33 (2020), 8765–8775.

[8] European Commission. 2019. ESCO handbook European skills, competences, qualifications and occupations. *Publications Office of the EU* (2019).

[9] Kushal Dave, Steve Lawrence, and David M Pennock. 2003. Mining the peanut gallery: Opinion extraction and semantic classification of product reviews. In *Proceedings of the 12th international conference on World Wide Web*. 519–528.

[10] Jens-Joris Decorte, Jeroen Van Hautte, Johannes Deleu, Chris Develder, and Thomas Demeester. 2023. Career path prediction using resume representation learning and skill-based matching. *arXiv preprint arXiv:2310.15636* (2023).

[11] Jens-Joris Decorte, Jeroen Van Hautte, Thomas Demeester, and Chris Develder. 2021. Jobbert: Understanding job titles through skills. *arXiv preprint arXiv:2109.09605* (2021).

[12] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2018. BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. *arXiv preprint arXiv:1810.04805* (2018).

[13] Maryam Fazel-Zarandi and Mark S Fox. 2009. Semantic matchmaking for job recruitment: an ontology-based hybrid approach. In *Proceedings of the 8th International Semantic Web Conference*, Vol. 525. 2009.

[14] Kaiming He, Haoqi Fan, Yuxin Wu, Saining Xie, and Ross Girshick. 2020. Momentum contrast for unsupervised visual representation learning. *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020), 9729–9738.

[15] Sarthak Jain and Byron C Wallace. 2019. Attention is not explanation. *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)* (2019), 3543–3556.

[16] Yannis Kalantidis, Mert Bulent Sariyildiz, Noe Pion, Philippe Weinzaepfel, and Diane Larlus. 2020. Hard negative mixing for contrastive learning. *Advances in Neural Information Processing Systems* 33 (2020), 21798–21809.

[17] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)* (2020), 6769–6781.

[18] Dor Lavi, Volodymyr Medentsiy, and David Graus. 2021. consultantbert: Fine-tuned siamese sentence-bert for matching jobs and job seekers. *arXiv preprint arXiv:2109.06501* (2021).

[19] Scott M Lundberg and Su-In Lee. 2017. A unified approach to interpreting model predictions. *Advances in neural information processing systems* 30 (2017).

[20] Stuart E Middleton, Nigel R Shadbolt, and David C De Roure. 2004. Ontological user profiling in recommender systems. In *ACM Transactions on Information Systems*, Vol. 22. ACM, 54–88.

[21] Chuan Qin, Hengshu Zhu, Tong Xu, Chen Zhu, Liang Jiang, Enhong Chen, and Hui Xiong. 2018. Enhancing person-job fit for talent recruitment: An ability-aware neural network approach. In *The 41st international ACM SIGIR conference on research & development in information retrieval*. 25–34.

[22] Marco Tulio Ribeiro, Sameer Singh, and Carlos Guestrin. 2016. "Why should I trust you?" Explaining the predictions of any classifier. *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (2016), 1135–1144.

[23] Joshua Robinson, Chia-Yuan Chuang, Suvrit Sra, and Stefanie Jegelka. 2021. Contrastive learning with hard negative samples. *International Conference on Learning Representations* (2021).

[24] Julian Rosenberger, Lukas Wolfrum, Sven Weinzierl, Mathias Kraus, and Patrick Zschech. 2025. CareerBERT: Matching resumes to ESCO jobs in a shared embedding space for generic job recommendations. *Expert Systems with Applications* 275 (2025), 127043.

[25] Aameek Singh, Carolyn Rose, Karthik Visweswariah, Vijil Chenthamarakshan, and Nandakishore Kambhatla. 2010. PROSPECT: A system for screening candidates for recruitment. In *Proceedings of the 19th ACM international conference on Information and knowledge management*. 659–668.

[26] Nasim Sonboli, Jessie J Smith, Florencia Cabral Berenfus, Robin Burke, and Casey Fiesler. 2021. Fairness and transparency in recommendation: The users' perspective. In *Proceedings of the 29th ACM Conference on User Modeling, Adaptation and Personalization*. 274–279.

[27] Afrina Tabassum, Muntasir Wahed, Hoda Eldardiry, and Ismini Lourentzou. 2022. Hard negative sampling strategies for contrastive representation learning. *arXiv preprint arXiv:2206.01197* (2022).

[28] Gaozhi Tang, Jianrong Wang, Xinglong Chang, Di Jin, and Xiudong Han. 2025. FitCLM: LLM-Augmented Candidate-aware Cross-view Contrastive Learning for Person-Job Fit. In *2025 28th International Conference on Computer Supported Cooperative Work in Design (CSCWD)*. IEEE, 2514–2520.

[29] Théo Trouillon, Johannes Welbl, Sebastian Riedel, Éric Gaussier, and Guillaume Bouchard. 2016. Complex embeddings for simple link prediction. *International conference on machine learning* (2016), 2071–2080.

[30] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. 2018. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).

[31] Hongwei Wang, Fuzheng Zhang, Xing Xie, and Minyi Guo. 2018. DKN: Deep knowledge-aware network for news recommendation. In *Proceedings of the 2018 world wide web conference*. 1835–1844.

[32] Xiang Wang, Xiangnan He, Yixin Cao, Meng Liu, and Tat-Seng Chua. 2019. KGAT: Knowledge Graph Attention Network for Recommendation. *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2019), 950–958.

[33] Xiang Wang, Xiangnan He, Meng Wang, Fuli Feng, and Tat-Seng Chua. 2019. Neural graph collaborative filtering. *Proceedings of the 42nd international ACM SIGIR conference on Research and development in Information Retrieval* (2019), 165–174.

[34] Sarah Wiegreffe and Yuval Pinter. 2019. Attention is not not explanation. *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)* (2019), 11–20.

[35] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. *International Conference on Learning Representations* (2021).

[36] Rex Ying, Ruining He, Kaifeng Chen, Pong Eksombatchai, William L Hamilton, and Jure Leskovec. 2018. Graph convolutional neural networks for web-scale recommender systems. *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining* (2018), 974–983.

[37] Xiao Yu, Jinzhong Zhang, and Zhou Yu. 2024. Confit: Improving resume-job matching using data augmentation and contrastive learning. In *Proceedings of the 18th ACM Conference on Recommender Systems*. 601–611.

[38] Fuzheng Zhang, Nicholas Jing Yuan, Defu Lian, Xing Xie, and Wei-Ying Ma. 2016. Collaborative knowledge base embedding for recommender systems. *Proceedings of the 22nd ACM SIGKDD international conference on knowledge discovery and data mining* (2016), 353–362.

[39] Mike Zhang, Rob Van Der Goot, and Barbara Plank. 2023. ESCOXLM-R: Multilingual taxonomy-driven pre-training for the job market domain. *arXiv preprint arXiv:2305.12092* (2023).

[40] Yongfeng Zhang and Xu Chen. 2020. Explainable recommendation: A survey and new perspectives. *Foundations and Trends® in Information Retrieval* 14, 1 (2020), 1–101.

A Research Methods

A.1 Part One

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Morbi malesuada, quam in pulvinar varius, metus nunc fermentum urna, id sollicitudin purus odio sit amet enim. Aliquam ullamcorper eu ipsum vel mollis. Curabitur quis dictum nisl. Phasellus vel semper risus, et lacinia dolor. Integer ultricies commodo sem nec semper.

A.2 Part Two

Etiam commodo feugiat nisl pulvinar pellentesque. Etiam auctor sodales ligula, non varius nibh pulvinar semper. Suspendisse nec lectus non ipsum convallis congue hendrerit vitae sapien. Donec at laoreet eros. Vivamus non purus placerat, scelerisque diam eu, cursus ante. Etiam aliquam tortor auctor efficitur mattis.

B Online Resources

Nam id fermentum dui. Suspendisse sagittis tortor a nulla mollis, in pulvinar ex pretium. Sed interdum orci quis metus euismod, et sagittis enim maximus. Vestibulum gravida massa ut felis suscipit congue. Quisque mattis elit a risus ultrices commodo venenatis eget dui. Etiam sagittis eleifend elementum.

Nam interdum magna at lectus dignissim, ac dignissim lorem rhoncus. Maecenas eu arcu ac neque placerat aliquam. Nunc pulvinar massa et mattis lacinia.

Received 20 February 2007; revised 12 March 2009; accepted 5 June 2009